

# flux2-image-generator

---

**Version:** 1.0.0 **Last Updated:** 2025-12-23

## Description

FLUX.2 [dev] FP8 Image Generation Skill - A comprehensive skill for generating high-quality images using the FLUX.2 [dev] model with FP8 quantization. This skill handles prompt optimization, parameter configuration, and production-grade image generation following Black Forest Labs' best practices.

## When to Use

Use this skill when:

- User requests image generation from text prompts
- User asks to create images with specific styles or technical requirements
- User wants to generate professional-quality images
- User mentions FLUX.2, text-to-image, or image generation
- User provides creative descriptions that should be turned into images

## Capabilities

This skill provides:

1. **Intelligent Prompt Enhancement:** Automatically optimizes user prompts following FLUX.2 best practices
2. **Parameter Configuration:** Selects optimal generation parameters based on use case
3. **Quality Tiers:** Fast preview, balanced production, or high-quality final renders
4. **Style Application:** Photography styles, lighting techniques, art styles, and color control
5. **Safety Compliance:** Content moderation and policy adherence
6. **Production Integration:** Ready for Modal + ComfyUI backend deployment

## Parameters

### Input Parameters

- **prompt** (required): The text description of the image to generate
- **quality\_tier** (optional): "preview", "balanced", or "final" (default: "balanced")
- **aspect\_ratio** (optional): "1:1", "16:9", "9:16", "3:2", "2:3", "4:3", "21:9" (default: "1:1")
- **style** (optional): "photorealistic", "artistic", "cinematic", "anime", "illustration", "3d-render"
- **camera\_spec** (optional): Camera and lens specification for photorealistic images
- **lighting** (optional): Lighting description for enhanced control
- **color\_palette** (optional): Color palette or hex codes for specific colors
- **seed** (optional): Seed for reproducible results
- **negative\_elements** (optional): Elements to avoid (will be converted to positive descriptions)

### Output

Returns a structured generation request with:

- Optimized prompt following FLUX.2 best practices
- Generation parameters (width, height, steps, guidance, sampler, scheduler)
- Cost estimate and generation time estimate
- Safety compliance status
- Ready-to-use API payload for backend

## Knowledge Base

This skill leverages comprehensive research on FLUX.2 [dev]:

### Model Specifications

- **Architecture:** 32B parameter rectified flow transformer
- **VRAM (FP8):** 12GB (optimized) to 32GB (raw)
- **Resolution Range:** 512×512 (0.5MP) to 2048×2048 (4MP)
- **Quantization:** FP8 (95%+ perceptual quality vs full precision)
- **License:** FLUX.2 [dev] Non-Commercial License

### Core Principles

1. **Natural Language Prompts:** FLUX.2 prefers conversational descriptions over keyword lists
2. **No Negative Prompts:** Always use positive descriptions
3. **Subject-First Structure:** Place main subject at beginning of prompt
4. **Distilled Guidance:** Uses FluxGuidance (2-4 range) instead of traditional CFG
5. **Layered Prompting:** Subject → Action → Environment → Style → Technical specs

### Parameter Recommendations

#### Quality Tiers

**Preview** (Fast iteration, 12-15s):

- Resolution: 1024×1024
- Steps: 12
- Guidance: 3.0
- Cost: ~\$0.010/generation

**Balanced** (Production default, 18-25s):

- Resolution: 1536×1024
- Steps: 30
- Guidance: 3.5
- Cost: ~\$0.017/generation

**Final** (Maximum quality, 30-40s):

- Resolution: 2048×2048
- Steps: 40

- Guidance: 4.0
- Cost: ~\$0.028/generation

## ComfyUI Configuration

- **Sampler:** euler (default)
- **Scheduler:** sgm\_uniform
- **CFG:** Must be 1.0 (use FluxGuidance instead)

## Prompting Techniques

### Prompt Structure Template

```
[Subject], [action/pose]
[Environment/setting details]
[Lighting description]
[Camera/technical specs]
[Style/mood]
[Color palette]
[Refinements]
```

## Photography Styles

"Shot on [Camera] [Lens], [Settings]"

Examples:

- "Shot on Fujifilm X-T5, 35mm f/1.4, shallow depth of field"
- "Shot on Canon EOS R5, 70mm f/2.8, natural lighting"
- "Vintage 1970s film photography on Kodak Portra 400"

## Lighting Techniques

- "Dramatic low-key lighting with single source from left"
- "Golden hour natural light, backlighting"
- "Soft diffused studio lighting, beauty dish"
- "Volumetric god rays, cinematic atmosphere"

## Art Styles

- "Studio Ghibli anime style, hand-drawn animation"
- "Impressionist oil painting, visible brushstrokes, Monet style"
- "Hyperrealistic 3D render, octane render, dramatic lighting"
- "Minimalist vector illustration, flat colors, geometric shapes"

## Color Control

Associate hex codes with objects:

- "The sports car is #FF0000 (bright red)"
- "Neon sign glowing #00FFFF (cyan) against dark wall"

Negative to Positive Conversion

| User Says "No..." | Convert to Positive                     |
|-------------------|-----------------------------------------|
| "no blur"         | "sharp focus throughout"                |
| "no people"       | "empty scene" or "solitary landscape"   |
| "no text"         | "clean design" or "minimal composition" |
| "no distortion"   | "accurate proportions"                  |

Content Safety

Built-in filters detect:

- NSFW content
- Public figure references
- Copyrighted characters/logos
- Harmful imagery

Implementation: Always validate prompts before generation and log outputs.

Tools Available

This skill has access to:

- Prompt analysis and optimization
- Parameter calculation based on quality tier and aspect ratio
- Cost and time estimation
- Safety compliance checking
- API payload generation for Modal backend

Usage Examples

Example 1: Simple Request

User: "Generate an image of a futuristic city at sunset"

Skill Output:

Optimized Prompt: "A futuristic cityscape at golden hour sunset, towering glass and steel skyscrapers with neon lights beginning to illuminate, flying vehicles in the distance, warm orange and pink sky with dramatic clouds, cyberpunk aesthetic, shot on Sony A7IV 24mm f/2.8, wide angle establishing shot, cinematic composition"

Parameters:

- Quality: balanced

- Resolution: 1536×1024 (3:2 landscape)
- Steps: 30
- Guidance: 3.5
- Estimated Time: ~20s (warm) / ~40s (cold)
- Estimated Cost: \$0.017

## Example 2: Photorealistic Portrait

User: "Create a professional headshot of a business executive"

Skill Output:

Optimized Prompt: "Professional corporate headshot of a confident business executive in formal attire, neutral gray background, facing camera with slight smile, shot on Canon EOS R5 85mm f/1.8, shallow depth of field with blurred background, soft diffused studio lighting from front with subtle rim light from behind, sharp focus on eyes, professional photography, high-end corporate style"

Parameters:

- Quality: final
- Resolution: 1024×1536 (2:3 portrait)
- Steps: 40
- Guidance: 4.0
- Style: photorealistic
- Estimated Time: ~35s
- Estimated Cost: \$0.024

## Example 3: Artistic Style

User: "Make an anime-style illustration of a magical forest"

Skill Output:

Optimized Prompt: "Enchanted magical forest in Studio Ghibli anime style, ancient trees with glowing bioluminescent mushrooms, ethereal spirits floating through shafts of sunlight filtering through canopy, vibrant greens and blues with warm golden light, hand-drawn animation aesthetic, painterly style, whimsical and dreamlike atmosphere, detailed foliage and magical atmosphere"

Parameters:

- Quality: balanced
- Resolution: 1920×1080 (16:9 widescreen)
- Steps: 30
- Guidance: 3.5
- Style: anime
- Estimated Time: ~22s
- Estimated Cost: \$0.018

## Example 4: Specific Technical Requirements

User: "I need a product photo of a red sports car, high quality, for web use"

Skill Output:

Optimized Prompt: "Professional automotive product photography of a sleek red sports car #DC143C (crimson), three-quarter front view angle, studio lighting setup with key light from front-left and rim lighting outlining the vehicle edges, black gradient background, sharp focus throughout entire vehicle, shot on Phase One XF IQ4 80mm f/5.6, commercial automotive photography, high-end magazine quality, reflective paintwork showing precise highlights"

Parameters:

- Quality: final
- Resolution: 2048×1536 (4:3)
- Steps: 40
- Guidance: 4.0
- Style: photorealistic
- Camera: Phase One XF IQ4 80mm
- Output Format: WebP (web optimized)
- Estimated Time: ~38s
- Estimated Cost: \$0.026

## Example 5: With Negative Elements

User: "Create a landscape with mountains, no people, no buildings, no text"

Skill Output:

Optimized Prompt: "Pristine wilderness landscape with majestic mountain peaks in the distance, untouched natural terrain with alpine meadows in foreground, empty scene showing only natural elements, crystal clear focus throughout, dramatic clouds rolling over mountain summits, golden hour lighting, shot on Hasselblad 500C medium format, vibrant natural colors, serene and solitary atmosphere"

Parameters:

- Quality: balanced
- Resolution: 2560×1080 (21:9 ultrawide)
- Steps: 30
- Guidance: 3.5
- Note: Converted negative prompts to positive descriptions
- Estimated Time: ~25s
- Estimated Cost: \$0.019

## Implementation Guidelines

### Prompt Optimization Algorithm

```
def optimize_prompt(user_input, style=None, camera_spec=None, lighting=None,
                    color_palette=None, negative_elements=None):
    """
```

```

Optimizes user prompt following FLUX.2 best practices
"""

# 1. Extract core subject (ensure it's first)
subject = extract_main_subject(user_input)

# 2. Convert negative elements to positive descriptions
if negative_elements:
    positive_additions = convert_negatives_to_positives(negative_elements)

# 3. Build layered prompt structure
prompt_layers = [
    subject, # Subject first (critical)
    extract_action_context(user_input),
    extract_environment(user_input),
    lighting or generate_lighting_for_style(style),
    camera_spec or generate_camera_for_style(style),
    style_description(style),
    color_palette,
    positive_additions
]

# 4. Combine with natural language (no keyword stuffing)
optimized = combine_natural_language(prompt_layers)

# 5. Validate length (30-80 words ideal)
if len(optimized.split()) < 30:
    optimized = expand_with_details(optimized)
elif len(optimized.split()) > 150:
    optimized = condense_to_essentials(optimized)

return optimized

```

## Parameter Selection Logic

```

def select_parameters(quality_tier, aspect_ratio, style):
    """
    Selects optimal generation parameters
    """

    # Quality tier configurations
    quality_configs = {
        "preview": {
            "steps": 12,
            "guidance": 3.0,
            "base_resolution": 1024
        },
        "balanced": {
            "steps": 30,
            "guidance": 3.5,
            "base_resolution": 1536
        }
    }

```

```

    },
    "final": {
        "steps": 40,
        "guidance": 4.0,
        "base_resolution": 2048
    }
}

config = quality_configs[quality_tier]

# Calculate resolution from aspect ratio
width, height = calculate_resolution(
    config["base_resolution"],
    aspect_ratio,
    ensure_divisible_by=32
)

# Adjust guidance for style
if style == "photorealistic":
    config["guidance"] = min(config["guidance"] + 0.5, 7.0)
elif style == "artistic":
    config["guidance"] = max(config["guidance"] - 0.5, 2.0)

return {
    "width": width,
    "height": height,
    "steps": config["steps"],
    "guidance": config["guidance"],
    "sampler": "euler",
    "scheduler": "sgm_uniform",
    "cfg": 1.0 # Always 1.0 for FLUX
}

```

## Safety Validation

```

def validate_safety(prompt):
    """
    Checks prompt for content policy violations
    """

    violations = []

    # Check for explicit NSFW keywords
    if contains_nsfw_keywords(prompt):
        violations.append("NSFW content detected")

    # Check for public figures
    if contains_public_figures(prompt):
        violations.append("Public figure reference")

    # Check for copyrighted content

```

```

if contains_copyrighted_content(prompt):
    violations.append("Copyrighted content")

return {
    "safe": len(violations) == 0,
    "violations": violations,
    "recommendation": "Rephrase prompt" if violations else "Approved"
}

```

## API Payload Generation

```

def generate_api_payload(optimized_prompt, parameters, seed=None):
    """
    Creates backend API request payload
    """

    return {
        "model": "flux2-dev-fp8",
        "prompt": optimized_prompt,
        "width": parameters["width"],
        "height": parameters["height"],
        "num_inference_steps": parameters["steps"],
        "guidance_scale": parameters["guidance"],
        "sampler": parameters["sampler"],
        "scheduler": parameters["scheduler"],
        "seed": seed or generate_random_seed(),
        "output_format": "png",
        "safety_check": True
    }

```

## Integration with Project

### Backend Integration (Modal)

This skill generates payloads compatible with the Modal backend architecture:

```

# modal_app/api.py endpoint
@app.function(gpu="A100-80GB", timeout=900)
def generate_image(request: ImageGenerationRequest):
    """
    Modal function for FLUX.2 FP8 image generation
    """

    # Load FLUX.2 FP8 model (lazy loading)
    load_flux2_fp8_if_needed()

    # Execute ComfyUI workflow with skill-generated params
    result = execute_comfyui_workflow(
        workflow_template="flux2_text2img.json",

```

```
    prompt=request.prompt,
    width=request.width,
    height=request.height,
    steps=request.num_inference_steps,
    guidance=request.guidance_scale,
    sampler=request.sampler,
    scheduler=request.scheduler,
    seed=request.seed
  )

  # Upload to R2
  output_url = upload_to_r2(result.image_data)

  # Emit completion event to Inngest
  emit_inngest_event("generation/completed", {
    "job_id": request.job_id,
    "output_url": output_url
  })

  return {"output_url": output_url}
```

## Frontend Integration (Next.js)

```
// Frontend API route usage
export async function POST(request: Request) {
  const { prompt, quality_tier, aspect_ratio, style } = await request.json()

  // Use skill logic to optimize
  const optimized = optimizePromptWithSkill(prompt, {
    quality_tier,
    aspect_ratio,
    style
  })

  // Submit to Inngest
  await inngest.send({
    name: "generation/requested",
    data: {
      userId: session.user.id,
      type: "image",
      model: "flux2-dev-fp8",
      prompt: optimized.prompt,
      parameters: optimized.parameters,
      estimatedCost: optimized.cost_estimate
    }
  })

  return Response.json({
    jobId: job.id,
    estimatedTime: optimized.time_estimate
  })
}
```

```
    })  
  }
```

## Performance Targets

- **Preview Quality:** <15s generation time, <\$0.01 cost
- **Balanced Quality:** <25s generation time, <\$0.02 cost
- **Final Quality:** <40s generation time, <\$0.03 cost
- **Prompt Optimization:** <100ms processing time
- **Safety Validation:** <50ms processing time

## Limitations

1. **Text Rendering:** FLUX.2 struggles with long text (>5 characters). Recommend post-processing for precise text.
2. **Public Figures:** Model filters public figure references (safety compliance).
3. **NSFW Content:** Built-in filters prevent generation of explicit content.
4. **Multi-Model Scenes:** Cannot combine multiple specific models/characters (copyright protection).
5. **Extreme Resolutions:** Limited to 4MP (2048×2048). Higher resolutions require upscaling.

## Error Handling

```
def handle_generation_errors(error):  
    """  
    Provides user-friendly error messages  
    """  
  
    error_map = {  
        "VRAMOutOfMemory": {  
            "message": "Resolution too high for available resources",  
            "suggestion": "Try reducing resolution or switching to 'preview'  
quality"  
        },  
        "SafetyFilterTriggered": {  
            "message": "Prompt violates content policy",  
            "suggestion": "Rephrase prompt to avoid restricted content"  
        },  
        "TimeoutError": {  
            "message": "Generation took too long",  
            "suggestion": "Reduce steps or resolution, or try again"  
        },  
        "ModelLoadError": {  
            "message": "Failed to load FLUX.2 model",  
            "suggestion": "Backend issue - please try again in a moment"  
        }  
    }  
  
    return error_map.get(error.type, {  
        "message": "Generation failed",
```

```
    "suggestion": "Please try again or contact support"
  })
```

## Future Enhancements

Planned improvements for v2.0:

- 1. **ControlNet Integration:** Pose control, depth maps, edge detection
- 2. **Multi-Image Reference:** Combine up to 10 reference images (FLUX.2 native capability)
- 3. **Inpainting/Outpainting:** Edit specific regions or expand canvas
- 4. **Batch Generation:** Generate multiple variations in one request
- 5. **Style Presets:** User-savable style configurations
- 6. **Advanced Scheduling:** Priority queue for pro users

## Related Skills

This skill works well with:

- [video-from-image](#): Convert generated images to videos using CogVideoX
- [image-upscaler](#): Enhance resolution beyond 4MP
- [style-transfer](#): Apply artistic styles to generated images
- [content-moderator](#): Enhanced safety filtering beyond built-in

## References

For detailed information, refer to:

- [FLUX2\\_FP8\\_COMPREHENSIVE\\_GUIDE.md](#)
- [PRD.md](#) - Project architecture
- [CLAUDE.md](#) - Development guidelines

## Changelog

**v1.0.0** (2025-12-23):

- Initial release
- Complete FLUX.2 [dev] FP8 implementation
- Production-ready prompt optimization
- ComfyUI + Modal integration
- Safety compliance framework
- Cost and performance optimization

---

**Maintained by:** AI Video Generation Platform Team **License:** Proprietary (aligned with FLUX.2 [dev] Non-Commercial License) **Last Tested:** 2025-12-23 with FLUX.2 [dev] FP8 on A100 80GB